your desk. (There is also a version called the EZ-LAB, using the ADSP-21062, that plugs into a slot in your computer). There are only four connections you need to worry about: DC power, a serial connection to your personal computer, and the input and output signals. A DC power supply and serial cable are even provided in the kit. The input and output signals are at audio level, about 1 volt amplitude. Alternatively, a jumper on the board allows a microphone to be directly attached into the input. The idea is to plug a microphone into the input, and attach a set of amplified speakers (such as used with personal computers) to the output. This allows you to *hear* the effect of various DSP algorithms.

Analog-to-digital and digital-to-analog conversion is accomplished with an Analog Devices AD1847 codec (coder-decoder). This is a 16 bit sigma-delta converter, capable of digitizing two channels (stereo) at a rate of up to 48k samples/second, and simultaneously outputing two channels at the same rate. Since the primary use of this board is to process audio signals, the inputs and outputs are AC coupled with a cutoff of about 20 Hz.

Three push buttons on the board allow the user to generate an interrupt, reset the processor, and toggle a flag bit that can be read by the system. Four LEDs mounted on the board can be turned on and off by toggling bits. If you are ambitious, there are sections of the board that allow you to access the serial port, link ports (only on the EZ-LAB with its ADSP-21062), and processor bus. However, these are *unpopulated*, and you will need to attach the connectors and other components yourself.

Here's how it works. When the power is applied, the processor boots from an on-board EPROM (512 kbytes), loading a program that establishes serial communication with your personal computer. Next, you launch the *EZ-Lite Host* program on you PC, allowing you to download programs and upload data from the DSP. Several prewritten programs come with the EZ-KIT Lite; these can be run by simply clicking on icons. For instance, a band-pass program allows you to speak into the microphone, and hear the result after passing through a band-pass filter. These programs are useful for two reasons: (1) they allow you to quickly get the system doing something interesting, giving you confidence that it does work, and (2) they provide a template for creating programs of your own. Which brings us to our next topic, a design example using the EZ-KIT Lite.

Design Example: An FIR Audio Filter

After you experiment with the prewritten programs for awhile, you will want to modify them to gain experience with the programming. Programs can be written in either assembly or C; the EZ-KIT Lite provides software tools to support both languages. Later in this chapter we will look at advanced methods of programming, such as *simulation*, *debugging*, and working in an *integrated development environment*. For now, we will focus on the easiest way to get a program to run. Little steps for little feet.